# Recover the Design: Solving Degree-Constrained Fence Layouts by Propagation and Backtracking

**Authors: Member A, Member B, Member C**

**Date: 2026-11-03**

# Chapter 1: Introduction

## 1.1 题目要求

题面给的是一张 $n$ 行 $m$ 列的棋盘（$1 < n, m \leq 50$）。每个格子要么是空的（记 0），要么放了一个“连接器”，格里写的 $k \in \{1, 2, 3, 4\}$ 就是这个连接器要接出去的栅栏条数。栅栏只能沿行或沿列、必须笔直，两端各落在一个连接器上；两个连接器之间如果还隔着别的连接器，就不能直接相连；两条栅栏不允许在没有连接器的格子里十字交叉（在没有连接器的格子里交叉等于两条线在同一个空格里叠着，题面明确禁止）。要求是：

1. 每个连接器接出的栅栏条数恰好等于 $k$；
2. 给出任意一个合法铺法；如果无解，输出一行 `No Solution`。

输出按行优先，每个连接器一行 6 个整数：`row col up down left right`，其中四个方向位是 0/1。题面给了三组样例：5×6 有一解；把第一组第 3 行改成 `2 2 1 0 0 1` 之后变成无解；3×5 的一组有两个解——这说明答案不唯一，任意合法解都算对。

本题 PR 阶段 20 分，构成是 Programming 10、Testing 3、Analysis 3、Documentation 4。本文档对应后 10 分（测试、分析、文档），代码在 `code/` 里。

## 1.2 背景：这个问题难在哪

先把三件事分清楚，后面选算法才不会走错路。

**第一，度数约束本身不难。**把每个连接器看成一个点，把“同行或同列相邻的两个连接器”看成一条候选边（下一节说明为什么只需相邻），那么“每个点恰好接 $k$ 条边”就是在这个图上找一个满足指定度数的子图。如果所有格子都是连接器、且 $k$ 全为 1，这就是“网格图上是否存在完美匹配”，多项式可解；更一般的 $k$ 对应 $f$-factor，Tutte 的 $f$-factor 定理给出多项式判定，也可以化成 $b$-matching 用网络流求。

**第二，让问题变难的是交叉约束。**一条横栅栏会压住它中间经过的每个空格，一条竖栅栏也一样，而同一个空格里横竖不能同时存在。这等于给“选哪些边”加了一批互斥对：横边与压在同一空格上的竖边最多选一条。每个空格最多只被一条横边和一条竖边压住，所以互斥关系是一个匹配（matching）——但“带互斥对的 $f$-factor ”没有现成的多项式算法，它是约束满足问题，最坏情况只能指数搜索。

**第三，样例的规模很小，但输入空间很大。**$n, m \leq 50$ 意味着最多 2500 个格子、约 5000 条候选边。要在 10 分的编程题里拿到分，光“能跑样例”不够：样例 1 的搜索树只走了一个结点，随便写个回溯都能过；真正决定分数的是那些把搜索树撑到几十万结点的输入（第 3 章有一整节专测）。

### 1.3 我们想回答的问题

1. 传播（propagation）能砍掉多少搜索？值不值得为它多写一百多行代码？
2. 分支点怎么挑？四种常见策略在同一批数据上差多少，有没有一种稳赢？
3. 50×50 的规模上限在哪——什么形状的输入最难，难到什么程度？
4. 为什么随机生成的度数表几乎不用搜索就能判出无解？

## Chapter 2: Data Structure / Algorithm Specification

### 2.1 从棋盘到图：建模

**点。**每个连接器是一个点，按行优先编号（这个编号顺序正好就是输出顺序，省掉一次排序）。

**边。**同一行（或同一列）里相邻的两个连接器之间，可以放一段栅栏，这就是一条候选边。为什么不连“隔着第三个连接器的两个点”？因为栅栏是直线，若它从 A 出发穿过中间的连接器 B 再到 C，那 B 也落在栅栏上，B 的度数算不算这条线就成了歧义；题面要求栅栏“两端是连接器”，所以一条栅栏的两端之间不允许再有连接器。这条限制把候选边的数量压到 $O(nm)$：每个点最多四个邻居（上下左右各一个），候选边最多 $2nm$。

**冲突。**一条横边会压住它中间经过的每个空格，一条竖边同理。同一个空格里如果同时有横边和竖边，就是题面禁止的交叉。所以冲突只发生在“压住同一个空格的横边与竖边”之间。因为每个空格最多被一条横边和一条竖边压住（边界上的空格更少），冲突图是若干个互不相连的二元组，这一点在第 2.3 节会被用来把检查做成 $O(1)$。

**形式的说法。**给定无向图 $G = (V, E)$（$V$ 是连接器，$E$ 是候选边）和度数向量 $k$，求 $S \subseteq E$，使得

$$\forall v \in V:\ |S \cap \delta(v)| = k(v), \qquad \forall (e_1, e_2) \in C:\ \neg\big(e_1 \in S \wedge e_2 \in S\big),$$

其中 $C$ 是冲突对集合。第一组约束是度数等式，第二组是互斥。

### 2.2 数据结构

表 1 列了求解器用到的全部数组。这里只说三个设计决定。

**决定一：冲突用“格子 → 边”反查，而不是枚举边对。**选一条横边时，要知道它压住的每个空格上有没有竖边。若拿边表两两比较，$E = 5000$ 时一次检查是 $10^7$ 量级；用 `hEdge_`/`vEdge_` 反查，每个空格一次数组访问，整条边走完就是它的长度。这条反查表在报告里只占两行代码，但它把“检查交叉”从搜索的热点里挪走了。

**决定二：状态用“改动日志”回滚，不整份拷贝。**一个搜索结点要能退回分支前的状态。

表 1: 求解器的核心数组（$V$ 为连接器数，$E$ 为候选边数）

| 名字 | 规模 | 作用 |
|---|---|---|
| `cid_` | $nm$ | 格子 → 连接器编号，$-1$ 表示空格 |
| `cellOf_` | $V$ | 连接器 → 格子编号（输出时反查坐标） |
| `edges_` | $E$ | 每条候选边的两个端点和两个端点坐标 |
| `inc_` | $2E$ | 连接器 → 关联的候选边（最多 4 条） |
| `hEdge_`, `vEdge_` | $nm$ | 空格 → 压住它的横/竖候选边，$-1$ 表示没有 |
| `st_` | $E$ | 每条边三态：0 未定 / 1 选中 / 2 弃用 |
| `need_` | $V$ | 这个点还差几条边 |
| `freeCnt_` | $V$ | 这个点还有几条未定的关联边 |
| `trail_` | 变长 | 改动日志，每条记录 { 何种量, 下标, 旧值 } |
| `queue_`, `inQueue_` | $V$ | 传播队列与去重标记 |

最直接的做法是进分支前把 `st_`/`need_`/`freeCnt_` 整份拷一份，$V+E$ 约 6000 个字段、每个 1–4 字节，一次拷贝几万字节；几十万结点就是几十 GB 的搬运，光 `memcpy` 就能吃掉全部时间预算。改成只记“哪个量被改成了什么”，一次传播通常只有几十条记录，回滚代价与“改了多少”成正比，与问题规模无关（表 1 里的 `trail_`）。代价是回滚必须严格倒序，所以所有写状态的代码都只走 `selectEdge`/`banEdge`/`addNeed`/`subFree` 四个入口，别处不许直写。

**决定三：传播用队列，不用“扫到不动点”。**一条边被选中会让两端的需求各减一，可能连锁触发邻居的推断；用队列按“谁被改了就把谁排进去”推进，每个点在一次传播里最多处理常数次（它的四条边被定死时各触发一次），比每轮全表扫一遍便宜。

### 2.3 三条传播规则

任何时刻，对每个连接器 $v$ 都有两个数：`need`[$v$]（还差几条）和 `free`[$v$]（还有几条未定边）。三条规则都是“必然的推断”，不是启发式：

```
propagate():                              -- 反复推进直到不动点
    while queue is not empty:
        v = pop()
        if need[v] < 0 or need[v] > free[v]: return FALSE
        if need[v] == 0:      ban  every undecided edge of v
        if need[v] == free[v]: select every undecided edge of v
    return TRUE
```

$need > free$ 说明剩下的边全选也不够，这一支必死；$need = 0$ 说明已经够了，未定的边一条都不能再选；$need = free$ 说明剩下的边一条都不能少。三条都是“合法解必须

满足”的条件，所以剪掉的分支里不可能有解——这一点在第 2.5 节证明。

选边本身也有两条后果，写成一个函数：

```
select(e):                          -- 选中一条边；FALSE 表示立刻矛盾
    if state[e] == SELECTED: return TRUE      -- 幂等
    if state[e] == BANNED:   return FALSE
    state[e] = SELECTED; need[u]--; need[v]--; free[u]--; free[v]--
    push u, push v                            -- 两端都要重新检查
    for each cell x strictly between the two endpoints:
        o = the perpendicular edge covering x  -- 由 hEdge/vEdge 直接查到
        if o exists:
            if state[o] == SELECTED: return FALSE   -- 已经交叉了
            ban(o)
    return TRUE
```

ban 与 select 对称：置状态、两端 free 减一、把两端排进队列。

**用样例 3 走一遍**。输入是 3 行 5 列：

```
1 1 2 0 1
1 0 3 0 2
1 2 1 0 1
```

右下角 (2,5)（1 基）的 $k = 2$，它的候选边只有“上”和“下”两条——同一列里相邻的连接器只有 (1,5) 和 (3,5)。于是 free=2、need=2，规则三把两条边都定死；这两条边一选，(1,5) 和 (3,5) 各消耗一个度，它们可能又满足“剩下的边一条都不能少”，继续往下推。整张图推完，只剩下两三个自由度，这时才进回溯。实测这张 3×5 的图 dfs 只走了 2 个结点就出了解（第 3 章样例一节）。

## 2.4 分支与回溯

传播停下来时，若还有 need>0 的点，就说明有自由度，需要枚举。分支点的挑法是一个启发式：

```
choose_vertex():                    -- 返回 -1 表示所有 need 都是 0，已经成功
    among v with need[v] > 0:
        pick the one minimizing (free[v] - need[v])   -- 最“被逼死”的点
        tie-break: minimizing free[v]                 -- 再挑边少的
```

这就是 CSP 里常说的 MRV（minimum remaining values）：先处理最没有余地的点，失败得早，剪枝早。分支宽度最多 4（一个点最多四条边），枚举“选其中 need 条”的全部

组合，最多 $C(4,2)=6$ 种。

```
dfs():
    nodes++
    if propagation is on and not propagate(): return FALSE
    v = choose_vertex()
    if v < 0: return TRUE                          -- 所有 need 归零，方案成立
    cand = undecided edges of v
    mark = trail.size()
    for each subset S of cand with |S| == need[v]:
        ok = TRUE
        for e in S:               ok = ok and select(e)
        for e in cand \ S:        ok = ok and ban(e)
        if ok and dfs(): return TRUE
        undo(mark)                                 -- 这一组失败，整层退回
    return FALSE
```

undo(mark) 把 trail_ 弹到长度 mark，倒序还原；同时清空传播队列（队列里的点属于被撤销的那一层，状态已经回去了，不能再按它们检查）。

### 2.5 正确性

**引理 1（传播保解）**三条规则剪掉的分支里没有合法解。

证明。设当前部分赋值 $S$ 是某个合法解 $S^*$ 的子集。(a) 若 need[$v$]>free[$v$]，则 $v$ 的未定边全选也不足 $k(v)$，与 $S^*$ 存在矛盾；(b) 若 need[$v$]$=0$，则 $S^*$ 不能再含 $v$ 的任何未定边，否则度数超 $k(v)$；(c) 若 need[$v$]=free[$v$]，则 $S^*$ 必须含 $v$ 的全部未定边，否则凑不满。select 里弃掉对向边同理：同一空格里横竖不能共存，$S^*$ 若含这条横边就必不含那条竖边。 □

**引理 2（分支完备）**分支点 $v$ 处，任何合法解在 $v$ 的未定边里选中的条数必然恰好等于 need[$v$]，所以它对应枚举中的某一个子集 $S$。

证明。合法解要满足 $v$ 的度数等式；已选中的边贡献了 $k(v)-$need[$v$] 条，还差 need[$v$] 条，只能从未定边里取。 □

**定理**算法返回 TRUE 时输出的方案合法；返回 FALSE 时问题无解。

证明。返回 TRUE 时，每个 select 都检查了度数不超 $k$ 与不交叉，结束时所有 need 为 0（否则 choose_vertex 会返回一个点继续分支），度数与交叉两条约束都满足。返回 FALSE 时，根结点的所有分支都被试过：每一层按引理 2 覆盖了全部可能，每个失败的分支按引理 1 不含解，故无解。 □

**终止性**分支前传播已推到不动点，此时对分支点有 $0<$need$<$free，所以“选 need 条

”与“弃掉其余”两边都非空，每层至少把一条边从“未定”变成定值；边数有限，深度不超过 $E$，搜索树有限。

### 2.6 为什么不用别的做法

**裸回溯**（不传播）：正确但慢得离谱。第 3.7 节有对照：30×30 的同一组数据，开传播 26 个结点，关掉传播 259063 个结点，差了四个数量级。

**化成 SAT/ILP 交给现成求解器**：可行，但课程要求自己实现，而且本题规模（5000 个布尔变量、几千条子句）用通用求解器反而更重；我们的传播规则本质上是单位传播（unit propagation）的特化版本，把“度数恰好”这类基数约束直接编进数据结构，省掉了转 CNF 的开销。

**网络流 / $b$-matching**：能处理度数约束，但处理不了交叉互斥——互斥把问题从“流”变成了“带冲突的选边”，没有现成的多项式算法（见 1.2）。这也是本题值得写一章的原因。

## Chapter 3: Testing

### 3.1 环境与口径

所有时间都在同一台机器上测：Windows 11、g++ 16.2.0（MinGW-W64，UCRT）、`-O2 -std=c++17`、CMake 4.4.3 + Ninja 1.13.2、Python 3.13.12（测试脚本）。计时用 `std::chrono::steady_cl` 只包住 `solve()`：读入与输出不算，否则 2500 个连接器的 `printf` 会把表冲得没法看。每组数据跑三次取最快的一次，单次上限 10 秒，超时的单独记，不混进中位数。秒数只在本文这批数据内可比，换机器或换优化级别都要重测。

测试分五层，从便宜到贵：题面样例（3 组）→ 边界用例（7 组，手写）→ 与暴力对拍（400 组随机小数据）→ 规模与策略实验（几百组 10×10 到 50×50）→ 最坏情况（高密度与不可解）。

### 3.2 题面样例

三组样例的输入输出都按题面原文存进 `code/samples/`，跑出来的结果与题面给的参考解逐行相同（用 `Compare-Object` 比较，差异行数 0）。样例 3 有两个解，我们给出的正好是题面列出的那一个。第二组输出 `No Solution`，与题面一致。

需要说明的是，“逐行相同”不是判题要求——多解时任意合法解都对——它只是说明搜索顺序恰好与出题人一致（分支启发式第一次命中的就是那条解）。

### 3.3 边界用例

手写 7 组，覆盖“空棋盘、单点、一对、奇总度、唯一交叉、十字、50×50 全空”这些角落。判定由 code/tools/check.py 完成：它按题面规则独立校验答案（每个连接器一行、方向位 0/1、度数等于 $k$、沿方向走到第一个非空格必须是连接器、对向必须互相声明、横竖不得交叉），而不是拿程序输出跟自己比。

表 2: 边界用例：输入、期望、实际

| 用例 | 规模 | 要点 | 期望 | 实际 |
|---|---|---|---|---|
| edge1-empty | 3×3 | 全 0，没有连接器 | 输出为空 | 空 ✓ |
| edge2-single | 2×2 | 只有一个 $k = 1$ 的点 | No Solution | No Solution ✓ |
| edge3-pair | 2×3 | 一行两个 $k = 1$ 的点 | 一条横线 | 合法 ✓ |
| edge4-odd | 2×3 | 三个 $k = 1$，总度数为奇 | No Solution | No Solution ✓ |
| edge5-cross | 3×3 | 上下左右各一个 $k = 1$，只能交叉 | No Solution | No Solution ✓ |
| edge6-plus | 3×3 | 中心 $k = 4$，四向各一个 $k = 1$ | 十字 | 合法 ✓ |
| edge7-zero50 | 50×50 | 全 0，压规模 | 输出为空 | 空 ✓ |

7 组全过。其中 edge5 是关键的一组：四条候选边两两不冲突的部分不存在，唯一的铺法会让两条线在中心交叉，程序必须判无解而不是硬凑一个。

### 3.4 与暴力对拍

**口径。**另写一个暴力求解器 code/tools/brute.py（不复用求解器任何代码）：枚举每条候选边选/不选，带“某点剩余可选边不够 $k$”的剪枝。随机数据分两类：layout（先随机铺互不相交的栅栏，再由端点算度数，保证有解）和 random（每格独立随机 $k$，绝大多数无解）。小规模（3×3 到 6×6）以暴力的结论为标准答案：暴力说无解，程序必须输出 No Solution；暴力说有解，程序必须给出一个通过 check.py 的合法解。另外每组再跑 20 个 20×20 与 50×50 的 layout 大构造（必可解），只验解的合法性。

**结果。**400 组抽查（暴力有解 200、无解 200）：0 处不符；20 组大构造：0 处不符。更早的 120、150、200 组轮次也都是 0。

### 3.5 规模实验

固定分支策略（$h_0$），在 $10 \times 10$ 到 $50 \times 50$ 的方阵上各跑 12 个种子，密度（非零格子的比例）取 0.15/0.25/0.35。表 3 是中位数与最大值。

表 3: 规模实验：`layout` 输入，12 个种子（时间单位 ms）

| 边长 | 密度 | 结点中位 | 结点最大 | 时间中位 | 时间最大 | 超时 |
|---|---|---|---|---|---|---|
| 10 | 0.15 | 1 | 1 | 0.00 | 0.00 | 0 |
| 10 | 0.25 | 2 | 3 | 0.00 | 0.01 | 0 |
| 10 | 0.35 | 2 | 9 | 0.01 | 0.01 | 0 |
| 20 | 0.15 | 1 | 3 | 0.01 | 0.01 | 0 |
| 20 | 0.25 | 2 | 10 | 0.01 | 0.01 | 0 |
| 20 | 0.35 | 8 | 29 | 0.01 | 0.03 | 0 |
| 30 | 0.15 | 2 | 12 | 0.01 | 0.01 | 0 |
| 30 | 0.25 | 7 | 19 | 0.01 | 0.03 | 0 |
| 30 | 0.35 | 26 | 133 | 0.03 | 0.09 | 0 |
| 40 | 0.15 | 2 | 9 | 0.01 | 0.02 | 0 |
| 40 | 0.25 | 16 | 110 | 0.03 | 0.09 | 0 |
| 40 | 0.35 | 120 | 2374 | 0.11 | 1.45 | 0 |
| 50 | 0.15 | 4 | 20 | 0.02 | 0.03 | 0 |
| 50 | 0.25 | 68 | 149 | 0.08 | 0.15 | 0 |
| 50 | 0.35 | 772 | 159984 | 0.69 | 67.81 | 0 |

三件事值得读出来。第一，中位数一直在毫秒以下，说明大多数输入传播一遍就推完了（结点数 1 到 26）。第二，最大值与中位数差三个数量级：50×50 密度 0.35 那一行，中位 0.69 ms，最坏 67.81 ms、结点 159984——同一条命令，换一个种子，时间差一百倍。第三，增长主要来自规模与密度两个方向叠加：10×10 无论密度都是 0.01 ms 以下，50×50 密度 0.35 才第一次出现秒级以下的“长尾”。

### 3.6 分支策略对比

四种策略（$h_0$ 自由度减需求最小、$h_1$ 行优先第一个、$h_2$ 需求最大、$h_3$ 未定边最多）在同一批 12 组数据上比结点数（表 4）。选择“结点数”而不是“秒数”作主指标，是因为它不受机器负载影响，换台机器还是这个数。

结论有三条，都不太好听：

1. 小规模上四种策略几乎一样（30×30 全部在 100 结点以内）。差别只在大实例上出现。

表 4: 分支策略：结点数（ms 为中位数，各跑三次取最快）

| 数据 | $h_0$ | $h_1$ | $h_2$ | $h_3$ | $h_0$ ms | $h_2$ ms | $h_3$ ms |
|---|---|---|---|---|---|---|---|
| 30×30 d0.35 #0 | 2 | 2 | 7 | 2 | 0.02 | 0.02 | 0.02 |
| 30×30 d0.35 #1 | 26 | 26 | 998 | 30 | 0.03 | 0.22 | 0.04 |
| 30×30 d0.35 #2 | 23 | 23 | 28 | 19 | 0.03 | 0.04 | 0.04 |
| 30×30 d0.35 #3 | 16 | 16 | 21 | 12 | 0.02 | 0.03 | 0.03 |
| 30×30 d0.35 #4 | 25 | 25 | 33 | 23 | 0.03 | 0.04 | 0.03 |
| 30×30 d0.35 #5 | 11 | 11 | 8 | 23 | 0.02 | 0.02 | 0.04 |
| 30×30 d0.35 #6 | 16 | 16 | 41 | 13 | 0.03 | 0.04 | 0.03 |
| 30×30 d0.35 #7 | 49 | 49 | 19 | 29 | 0.04 | 0.03 | 0.05 |
| 50×50 d0.40 #0 | 28409 | 28409 | 87 | 381030 | 11.02 | 0.10 | 175.35 |
| 50×50 d0.40 #1 | 5896 | 5896 | 438 | 229639 | 3.79 | 0.31 | 160.41 |
| 50×50 d0.40 #2 | 72611 | 72611 | 超时 | 5891 | 29.44 | 超时 | 3.97 |
| 50×50 d0.40 #3 | 102490 | 102490 | 930 | 234 | 49.06 | 0.91 | 0.30 |
| 合计结点 | 209574 | 209574 | 2610 | 616945 | | | |

2. $h_2$（需求最大）在 50×50 上总体最好：合计结点 2610，是 $h_0$ 的 1/80；四组里三组快 30 到 100 倍。但它在 #2 上直接超时，而 $h_0$ 那组只要 29 ms——单一策略没有稳赢。

3. $h_3$（未定边最多）最不稳：既能 0.30 ms 解决 #3，也能在 #0 上烧掉 175 ms、38 万结点。

所以默认值仍取 $h_0$：它不是最快的，但它在这 12 组里没有一次爆掉，最坏值最小。想要更快，正确做法不是换一个更好的策略，而是“几个策略轮流试、谁先出解算谁”（第 4.5 节）。

### 3.7 传播值多少

把传播关掉（保留选边时的逐边检查），其余不变，同一批数据再跑一遍（表 5）。

最极端的一组是 30×30 #1：26 个结点对 259063 个结点，差四个数量级；两组 50×50 在关掉传播后 10 秒都跑不完。这一节回答第 1.3 节的第一个问题：传播那一百多行代码，买到的是“能不能在时限内跑完”，不是常数优化。

表 5: 传播开/关对照（$h_0$，超时上限 10 s）

| 数据 | 开：结点 | 开：ms | 关：结点 | 关：ms |
|---|---|---|---|---|
| 30×30 d0.35 #0 | 2 | 0.02 | 102 | 0.03 |
| 30×30 d0.35 #1 | 26 | 0.03 | 259063 | 35.20 |
| 30×30 d0.35 #2 | 23 | 0.03 | 47580 | 7.81 |
| 30×30 d0.35 #3 | 16 | 0.02 | 904 | 0.15 |
| 50×50 d0.40 #0 | 28409 | 10.29 | 超时 | 超时 |
| 50×50 d0.40 #1 | 5896 | 3.80 | 超时 | 超时 |

### 3.8 最坏情况

**随机 50×50**。40 组随机密度（0.2/0.3/0.4/0.5）的 `layout` 输入：全部有解、无超时，中位 0.19 ms，最坏 699.61 ms（$d = 0.50$ 的第 29 个种子，1138066 个结点、66079435 次传播）。这一个种子就占了 40 组总时间的绝大部分。

**高密度**。密度再往上走（0.55、0.60），坑位互相牵连，传播推不动，搜索树开始指数爆炸（表 6）。

表 6: 高密度 50×50：每组 8 个种子、单次上限 10 s

| 密度 | 跑完/总数 | 中位 ms | 最大 ms | 最大结点 | 超时 |
|---|---|---|---|---|---|
| 0.55 | 8/8 | 474.4 | 4438.3 | 11561381 | 0 |
| 0.60 | 5/8 | 1.3 | 213.4 | 389751 | 3 |

0.55 那一行没有超时，但最大一组走了 1156 万结点、4.4 秒；0.60 有 3 组 10 秒没跑完，而跑完的 5 组里中位数只有 1.3 ms——不是“密度越高越慢”这么简单，而是“高密度里有一小撮输入会把搜索树撑爆”。

**不可解实例**。50×50 每格以概率 $p$ 独立给一个随机 $k$（$p = 0.5/0.7/0.9$，各 15 组），结果全部判为无解、全部在 0 个结点内返回、时间 0.00 ms。原因不神秘：随机给的 $k$ 与邻居数对不上（比如角上给 $k = 4$），`need>free` 在根结点就成立，或者总度数是奇数，两条预检直接拦下（第 4.3 节展开）。这也说明“随机对拍”里 `random` 那一路其实很少真正考验搜索。

### 3.9 测试过程中修掉的问题

测试不只是“跑一遍看对不对”。表 7 是这轮里真正抓到的三个问题，前两个在校验/生成工具里，第三个在求解器里。

表 7: 测试中发现的三个问题（目的 / 期望 / 实际 / 可能原因 / 状态）

| 目的 | 期望 | 实际 | 可能原因 | 状态 |
|---|---|---|---|---|
| 校验“对向边也要声明” | 合法解全部通过 | 300 组里 104 组被误判为非法 | `check.py` 用 `in` 比较值而不是下标，下标 2/3 的反向永远不命中 | 已修 |
| 生成“必有解”的 `layout` 实例 | 生成器造出的实例一定有解 | 50×50 里 12/20 报无解 | 生成器只查 `owner==EMPTY`，而连接器格的 owner 也是 EMPTY，栅栏会从连接器身上穿过去 | 已修 |
| 对比四种分支策略 | 四种都给同一个解 | $h_2$/$h_3$ 输出全 0、根结点即返回成功 | 两种“越大越好”的策略误用了降序变量 `bestSlack`（初值 $10^9$），条件永假，`chooseVertex` 返回 $-1$ | 已修 |

三个问题的共同点是：*都是“工具说错了”而不是“程序算错了”*，而且都只有在“换一批数据”或“换一个策略”时才暴露。第二个尤其值得记：生成器与校验器如果共享同一个错误假设，对拍会一致地给出错误结论；所以 `layout` 的“必有解”是由构造方法保证的（先铺线再算度数），不是由生成器自己声称的。

修完之后重跑：400 组对拍 0 差异、7 组边界全过、四种策略在 120 组小数据上答案一致。

## Chapter 4: Analysis and Discussion

### 4.1 时间与空间复杂度

**建图** $O(nm + E)$：扫一遍格子认出连接器，再对每行每列扫一遍连相邻对，边表与反查表各填一次。

**一次传播**：队列里每个点被处理常数次（它的每条边被定死时触发一次，最多 4 次），每次处理看它不超过 4 条关联边，所以一次传播是 $O(V)$ 量级；但 `select` 走一遍边的跨度，跨度的总和受限于“每条边覆盖的空格数”，最坏 $O(nm)$。

**搜索**：设搜索树有 $T$ 个结点，每个结点要做一次传播加一次分支点选取，总时间 $O(T \cdot (V + \text{跨度}))$；$T$ 最坏是指数的（1.2 节已说明这是带互斥的 $f$-factor，没有已知多项式算法）。实测 $T$ 的分布极不均匀：多数输入 $T \le 30$，最坏的 50×50 高密度输入 $T$ 到 $10^7$。

**空间** $O(nm + E)$：所有数组都是问题规模的线性函数，没有递归栈之外的动态分配——回溯是手写栈，不是系统递归（`dfs` 递归深度不超过 $E$，5000 层，每层几十字节，也是在 KB 量级）。50×50 的最坏情况实测常驻内存不到 10 MB。

### 4.2 传播为什么这么有效

表 5 的数字可以解释成一句话：*这道题的约束是“局部紧”的*。一个连接器的度数只有 1 到 4，而它的邻居最多四个，所以“剩下的边够不够”这个判断在它身上几乎总是立刻出结论；一旦某条边被定死，它两端的自由度各减一，很可能立刻满足 $need = 0$ 或 $need = free$，连锁反应一路推下去。样例 3 是极端例子：8 个连接器，传播一步没分支就全定了。

反过来说，传播推不动的地方就是难的地方：需要“同时”决定一批互相牵连的边，任何单点推断都下不了结论。50×50 密度 0.6 的那批输入正是这样——大量 $k = 3, 4$ 的点彼此相邻，每条边的选择都同时影响两端的余量，传播停在“每个点都还有 2 到 3 种可能”，只能靠搜索。

### 4.3 为什么随机无解实例秒出

随机给 $k$ 的输入几乎必然违反两条必要条件：(a) 某点的 $k$ 大于它的邻居数（角上最多 2 个邻居，边上 3 个，内部 4 个）；(b) 总度数是奇数。50×50 时内部点占九成以上，随机 $k \in \{1..4\}$ 里 $k = 4$ 的点只要有一个邻居是空格就立刻违反 (a)；即使侥幸通过，总度数的奇偶也有一半概率不对。实测 $p = 0.5$ 时 15 组全在根结点返回，就是这个原因。

这也解释了为什么 `stress` 里 `random` 那一路虽然数量占一半，却很少真正压到搜索：它们几乎都在预检或第一层传播就死了。真正考验搜索的是 `layout` 造的“有解但紧”的实例。

### 4.4 与理论算法的对照

这张表想说清楚一件事：本题的难度不在“度数约束”（那是多项式可解的），而在“度数约束 + 几何互斥”这个组合。把已有的多项式算法套上来，缺的正是互斥那一条；把通用 SAT 求解器套上来，多的正是“转格式 + 通用性”的开销。定制传播 + 回溯是这两者之间的折中。

表 8: 几种做法在本题上的可行性

| 做法 | 复杂度 | 在本题上的问题 |
|---|---|---|
| 完美匹配（$k \equiv 1$、全格连接器） | 多项式（开花算法） | 只覆盖 $k = 1$ 且无空格的特例 |
| $f$-factor / $b$-matching | 多项式 | 处理不了交叉互斥 |
| SAT / ILP | 最坏指数，工程上很强 | 要转成 CNF/线性约束，5000 变量规模下反而不如定制传播轻 |
| 传播 + 回溯（本文） | 最坏指数,实测多数毫秒级 | 高密度输入有指数尾巴（表 6） |

### 4.5 可以怎么改进

1. **策略组合而不是单选。**表 4 显示 $h_2$ 总体最好但会爆，$h_0$ 稳但慢。做法是给每个结点设一个结点预算，超了就换下一个策略重来（重启），或者按数据特征（密度、$k = 4$ 的比例）选初始策略。改动很小，能把最坏情况从“超时”压到“几个策略里最好的那个”。

2. **学习失败原因。**现在失败只回滚，不记“为什么失败”。经典 CDCL 的做法是记下导致冲突的那组赋值，回溯时直接跳过多层（non-chronological backtracking）。本题的冲突几乎都来自“某点度数凑不齐”，可以记成“这组边里最多选 $j$ 条”的基数约束，下次直接剪。

3. **预处理收缩。**$k$ 等于 0/1 或邻居数的点可以先定死（传播已经做了大部分）；对“一条链上全是 $k = 2$ 的点”这类结构，可以整段决定，不必逐点分支。

4. **位并行。**状态是 0/1/2 三态,可以用两个 bit 存一条边,一次处理 32 条;本题 $E \leq 5000$，位并行能把每个结点的常数再压一个数量级，但不改变指数尾部。

### 4.6 结论

1. 把棋盘化成“连接器 + 相邻候选边 + 交叉互斥”之后，传播三条规则加 MRV 分支能在毫秒级解决 50×50 的多数实例；三组题面样例逐行一致，400 组随机对拍与 7 组边界全过。

2. 传播是这道题的胜负手：关掉它，30×30 的结点数从 26 涨到 259063，50×50 两组直接超时。

3. 分支策略没有稳赢：$h_2$ 在 50×50 上总体快 80 倍却会单组超时，默认仍取最稳的 $h_0$；想要更稳应当做策略组合（重启），而不是找一个“更好的”单策略。

4. 难度集中在高密度输入：密度 0.55 时最大 4.4 秒、1156 万结点，0.60 时有 3/8 超

时。这是本题真正的规模上限，也是 10 分编程部分唯一可能丢分的地方。

**局限。**第一，时间数字是单机单批次的，只作批内比较。第二，测试数据全是我们自己生成的（`layout` 与 `random` 两种），没有官方隐藏测试，真实评测里的数据分布可能不同。第三，最坏情况只测到 50×50；题目规模上限就是 50×50，但如果允许更大，指数尾部会更早出现。第四，没有做“策略组合”的对照实验，4.5 节第 1 条只是根据表 4 的推断，不是实测结论。

## References


[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, “Introduction to Algorithms”, MIT Press, 3rd edition, (2009)

[2] W. T. Tutte, “A short proof of the factor theorem for finite graphs”, Canadian Journal of Mathematics, 6:347–352, (1954)

[3] J. Edmonds, “Paths, trees, and flowers”, Canadian Journal of Mathematics, 17:449–467, (1965)

[4] A. Biere, M. Heule, H. van Maaren, T. Walsh (eds.), “Handbook of Satisfiability”, IOS Press, 2nd edition, (2021)

[5] R. Dechter, “Constraint Processing”, Morgan Kaufmann, (2003)

[6] S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, Pearson, 4th edition, (2020)

## Author List

| 成员 | 负责的内容 |
| --- | --- |
| 成员 A | 建模、传播与回溯求解器、命令行与输出 |
| 成员 B | 暴力对拍程序、随机数据生成器、结果校验器 |
| 成员 C | 规模与策略实验、报告撰写与版式、提交包整理 |

## Declaration

We hereby declare that all the work done in this project titled “Recover the Design: Solving Degree-Constrained Fence Layouts by Propagation and Backtracking” is of our independent effort as a group.

## Signatures

成员 A：_____ 成员 B：_____

成员 C：_____

# Chapter Appendix A: Source Code

源码在提交包的 `code/src/` 下，三个文件、共 604 行，其中纯注释 200 行。按课程要求统计的注释率见表 9（口径：纯注释行 = 只含注释的行；宽口径 = 纯注释行加行尾注释行；分母为文件总行数）。

表 9: 源码注释率（`python code/tools/stats.py` 统计）

| 文件 | 总行 | 纯注释 | 行尾注释 | 纯注释率 | 宽口径 |
|---|---|---|---|---|---|
| `main.cpp` | 102 | 31 | 1 | 30.4% | 31.4% |
| `solver.cpp` | 380 | 116 | 9 | 30.5% | 32.9% |
| `solver.h` | 122 | 53 | 30 | 43.4% | 68.0% |
| 合计 | 604 | 200 | 40 | 33.1% | 39.7% |

三个文件都过 30% 的线；按“纯注释行 / 代码行”算则是 58.1%。下面按“看代码的顺序”排：先看头文件的接口与不变量，再看求解器实现，最后看主程序的输入输出。

## A.1 `solver.h`

```cpp
// solver.h —— "Recover the Design" 的求解器接口
//
// 建模：把每个连接器看成一个点，同一行（或同一列）里**相邻**的两个连接器之间
// 可以铺一条栅栏，看成一条候选边。为什么只连相邻的：栅栏不能拐弯，一条栅栏的
// 两端必须落在连接器上；如果它中途经过第三个连接器，那条栅栏就只能在那里断开成
// 两条，于是任何合法方案都可以拆成"相邻连接器之间的一段一段"。
//
// 在这个模型下，问题变成：从候选边里选一个子集，使得
//   (1) 每个连接器 incident 的选中边数恰好等于它的 k；
//   (2) 选中的水平边和垂直边不能在空格处十字相交。
// 两条约束都是局部的，但叠在一起就是 NP 难的（含最大独立集作为特例），
// 所以这里用"约束传播 + 回溯"，并在回溯时优先挑最紧的点分支。
//
// 规模：连接器数 V ≤ nm，候选边数 E ≤ 2nm（每行/列相邻点各一条），
// 每点最多 4 条关联边。空间 O(nm + E)，建图 O(nm + E)。
//
// 几条贯穿全类的不变量，改代码时务必保持：
//   * st_[e] == 1 的边，两端 need_ 已经减过；st_[e] == 2 的边两端 freeCnt_ 已经减过，
//     所以"未定边数"和"还差几条"永远互相自洽，传播规则才有意义；
//   * 任何一次状态改动都必须在 trail_ 上留一条旧值，否则 undo 之后状态就脏了；
//   * queue_ 里只放"need_ 或 freeCnt_ 刚变过"的点；undo 会同时清空队列和 inQueue_；
//   * need_ ∈ [0, freeCnt_] 是搜索继续的前提，一旦越界 propagate() 立刻返回 false。
#pragma once

#include <array>
#include <vector>

namespace p3 {

// 一条候选边：两个连接器之间的一段水平或垂直栅栏。
```

```
// 存两端的格子坐标而不是只存连接器编号，是因为最后要按方向输出 1/0，
// 而且"这条边横跨了哪些格子"要靠坐标现算，省一份内存。
struct Edge {
    int u, v;              // 两端连接器的编号（0 基，按行优先编号）
    int r1, c1, r2, c2;    // 两端所在的格子（0 基）
};

class Solver {
public:
    // 传入 n 行 m 列的度数表（0 表示该格没有连接器），建图。
    // 这个接口永远返回 true：候选边怎么建都建得出来，真正的"无解"判断留给 solve()，
    // 因为"某个点边数不够"这类结论在 solve() 一开始就能统一处理，不必分两处写。
    bool init(int n, int m, const std::vector<int>& grid);

    // 搜索一个合法方案；返回 false 表示无解。
    // 可以重复调用（状态在函数开头会重置），但一次对象只对应一组输入。
    bool solve();

    // 以下接口只在该次 solve() 返回 true 后有效，且都返回内部引用，不拷贝。
    const std::vector<Edge>& edges() const { return edges_; }        // 全部候选边
    const std::vector<char>& chosen() const { return st_; }          // 每条边的状态
    const std::vector<int>& connectorCell() const { return cellOf_; }  // 连接器所在的格子

    // 把选中的边整理成题目要的方向表：第 i 项是第 i 个连接器的 {上,下,左,右}。
    // 连接器本身就按行优先编号，所以这个数组的顺序就是输出的顺序。
    std::vector<std::array<int, 4>> directions() const;

    // 供报告里的实验用：搜索树结点数、传播中处理过的点次数。
    // 两个计数都只增不减，solve() 开头清零，不影响任何判定逻辑。
    long long nodes() const { return nodes_; }
    long long propagations() const { return propagations_; }

    // 分支点的选取策略，只影响搜索快慢，不影响答案对不对：
    //   0 = 自由度减需求最小（默认，最接近被逼死的点）
    //   1 = 行优先的第一个点（最朴素）
    //   2 = 需求最大的点
    //   3 = 未定边最多的点
    // 第 4 章拿这四种在同一批数据上比结点数：小规模几乎一样，高密度大实例上
    // 2 的总结点数少两个数量级，但个别实例会反过来爆掉；3 最不稳。单一一句话
    // 选不出赢家，所以默认仍用最稳的 0。
    void setHeuristic(int h) { heur_ = h; }

    // 关掉传播（只保留逐边检查），用来量"传播到底值多少"。
    // 关掉后算法退化成一个带前向检查的回溯，仍然正确，只是结点数会暴涨。
    void setPropagation(bool on) { useProp_ = on; }

private:
    // ---- 建图 ----
    // 加一条候选边，并把它横跨的空格登记到 hEdge_/vEdge_。
    void addEdge(int u, int v, int r1, int c1, int r2, int c2);

    // ---- 状态修改（全部写 trail，可回滚）----
    void pushVertex(int v);                  // 把点排进待传播队列（去重）
    bool selectEdge(int e);                  // 选中一条边，连带否掉交叉的对向边
    bool banEdge(int e);                     // 弃用一条边（幂等）
    bool addNeed(int v, int delta);          // 改"还差几条"，负数即失败
```

```
    void subFree(int v);                    // 未定边数减一，并触发该点重新传播
    void undo(size_t mark);                 // 回滚到 trail 的某个长度

    // ---- 传播与搜索 ----
    bool propagate();                       // 反复套三条传播规则直到不动点
    bool dfs();                             // 回溯主体
    int chooseVertex() const;               // 挑下一个分支点，-1 表示全部满足

    // ---- 建图结果（solve() 之后只读）----
    int n_ = 0, m_ = 0;
    std::vector<int> deg_;       // 每个格子的 k，0 表示没有连接器
    std::vector<int> cid_;       // 格子 -> 连接器编号，-1 表示空
    std::vector<int> cellOf_;    // 连接器 -> 格子编号
    std::vector<Edge> edges_;    // 候选边表
    std::vector<std::vector<int>> inc_;   // 连接器 -> 关联的候选边
    std::vector<int> hEdge_, vEdge_;      // 空格 -> 盖住它的水平/垂直候选边

    // ---- 搜索状态（会被 undo 反复改写）----
    std::vector<char> st_;       // 每条边：0 未定 / 1 选中 / 2 弃用
    std::vector<int> need_;      // 每个连接器还差几条边
    std::vector<int> freeCnt_;   // 每个连接器还有几条未定的关联边
    std::vector<int> queue_;     // 待检查的连接器
    std::vector<char> inQueue_;  // 是否已在队列里，用来给 pushVertex 去重

    // 回溯用：只记"哪个量被改成了什么"，undo 时倒着还原。
    struct Change { int kind; int idx; int old; };
    std::vector<Change> trail_;

    // ---- 实验统计 ----
    long long nodes_ = 0;          // dfs() 的调用次数，即搜索树结点数
    long long propagations_ = 0;   // 传播里检查过的点次数
    int heur_ = 0;                 // 分支点策略，见 setHeuristic
    bool useProp_ = true;          // 是否开传播，见 setPropagation
};

} // namespace p3
```

## A.2 solver.cpp

```
// solver.cpp —— 求解器的实现
//
// 整体思路（细节见 solver.h 的建模说明）：
//   把"选哪些栅栏段"写成一个 0/1 约束满足问题，然后用"约束传播 + 回溯"去解。
//   传播负责把能确定的边一次定死（某点已经够了 / 剩下的边一条都不能少 / 被交叉挡住），
//   回溯只负责在剩下的自由度里挑一个点枚举。所有状态改动都压在 trail_ 里，
//   失败时按记录倒着还原，因此不需要在递归里拷贝状态。
//
// 下面几处是容易写错的地方，先记在这：
//   1. 候选边只连"同一行/列里相邻的两个连接器"。理由是一条栅栏不能拐弯，
//      如果它中途经过第三个连接器，那条线就只能在那里断开成两段。
//   2. 交叉约束是"边与边"的：同一格里最多一条横边一条竖边，所以查 hEdge_/vEdge_
//      就能唯一定位到要否掉的那条边，不用扫全图。
//   3. 传播里的三条规则都是"只删掉不可能的赋值"，不会删掉任何一个可行解，
//      这一点是回溯正确性的前提；反过来，分支枚举覆盖了该点所有可能的选法，
//      所以只要搜索失败，就说明真的无解。
```

```cpp
//
// 复杂度：建图 O(nm + E)，空间 O(nm + E)，E 是候选边数，最多 2nm。
// 搜索最坏是指数的（问题本身 NP 难），但传播在随机数据上几乎能一路推到底。
//
// 举个小例子说明传播怎么工作（题面样例 3，3×5）：
//   1 1 2 0 1
//   1 0 3 0 2
//   1 2 1 0 1
//   一开始每个点都按 k 算需求。右下角 (2,5) 的 k=2，但它的候选边只有"上"和"下"
//   两条（同一列里相邻的连接器只有上下两个），于是 free == need == 2，
//   规则三把两条边都定死；这两条边一选，(1,5) 和 (3,5) 的需求各减 1，
//   它们可能又满足"剩下的边一条都不能少"，连锁推下去，很多点不用分支就定了。
//   传播停下时还剩少数几个自由度，这时才进 dfs 枚举——这就是它比裸回溯快的原因。
#include "solver.h"

#include <algorithm>

namespace p3 {

namespace {
// trail 里记的三种量。用整数标签而不是三个 vector，是因为回溯时要按"改动的先后"
// 倒着还原，混在一个栈里最省事，也最不容易漏。
enum Kind { K_STATE = 0, K_NEED = 1, K_FREE = 2 };
}  // namespace

// 建图。做三件事：给连接器编号、连出候选边、记下每条边压住了哪些空格。
// 连接器按行优先编号，这样最后输出时不用再排序，编号顺序就是题目要的顺序。
bool Solver::init(int n, int m, const std::vector<int>& grid) {
    n_ = n;
    m_ = m;
    deg_ = grid;

    // 第一遍：给每个连接器编号。cid_ 是"格子 -> 编号"的反查表，-1 表示空格。
    cid_.assign((size_t)n_ * m_, -1);
    cellOf_.clear();
    for (int r = 0; r < n_; ++r) {
        for (int c = 0; c < m_; ++c) {
            if (deg_[r * m_ + c] == 0) continue;
            cid_[r * m_ + c] = (int)cellOf_.size();
            cellOf_.push_back(r * m_ + c);
        }
    }

    edges_.clear();
    hEdge_.assign((size_t)n_ * m_, -1);
    vEdge_.assign((size_t)n_ * m_, -1);
    inc_.assign(cellOf_.size(), {});

    // 第二遍：每一行里，相邻的两个连接器之间可以铺一条水平栅栏。
    // 中间没有别的连接器，所以这一段是"整段"，不会和同一行里别的候选边重叠。
    for (int r = 0; r < n_; ++r) {
        int lastC = -1;
        for (int c = 0; c < m_; ++c) {
            int id = cid_[r * m_ + c];
            if (id < 0) continue;
            if (lastC >= 0) addEdge(cid_[r * m_ + lastC], id, r, lastC, r, c);
```

```
            lastC = c;
        }
    }
    // 第三遍：每一列同理，竖着扫一遍。
    for (int c = 0; c < m_; ++c) {
        int lastR = -1;
        for (int r = 0; r < n_; ++r) {
            int id = cid_[r * m_ + c];
            if (id < 0) continue;
            if (lastR >= 0) addEdge(cid_[lastR * m_ + c], id, lastR, c, r, c);
            lastR = r;
        }
    }
    return true;
}

// 加一条候选边，并把它压住的空格登记到 hEdge_/vEdge_。
// 这两张表是"格子 -> 盖住它的候选边"，选边时靠它 O(1) 找到该否掉的对向边。
void Solver::addEdge(int u, int v, int r1, int c1, int r2, int c2) {
    edges_.push_back({u, v, r1, c1, r2, c2});
    const int e = (int)edges_.size() - 1;
    inc_[u].push_back(e);
    inc_[v].push_back(e);

    // 只登记"中间的空格"：两端是连接器，那条边本身不占用连接器所在的格子。
    if (r1 == r2) {
        const int lo = std::min(c1, c2), hi = std::max(c1, c2);
        for (int c = lo + 1; c < hi; ++c) hEdge_[r1 * m_ + c] = e;
    } else {
        const int lo = std::min(r1, r2), hi = std::max(r1, r2);
        for (int r = lo + 1; r < hi; ++r) vEdge_[r * m_ + c1] = e;
    }
}

// 把点排进待检查队列。同一个点在一次传播里只排一次队，否则队列会被重复项撑大。
void Solver::pushVertex(int v) {
    if (inQueue_[v]) return;
    inQueue_[v] = 1;
    queue_.push_back(v);
}

// 改一个点"还差几条边"的计数。先记旧值再改，保证 undo 能还原。
// 返回 false 表示需求变成负数——说明有人被安排了比 k 还多的边，这一支已经死了。
bool Solver::addNeed(int v, int delta) {
    trail_.push_back({K_NEED, v, need_[v]});
    need_[v] += delta;
    return need_[v] >= 0;
}

// "还剩下几条未定的关联边"减一。这个量本身不参与判定，只用来算自由度和挑分支点，
// 但它一变，该点的三条传播规则就可能适用，所以要把它排进队列。
void Solver::subFree(int v) {
    trail_.push_back({K_FREE, v, freeCnt_[v]});
    --freeCnt_[v];
    pushVertex(v);
}
```

```
129
130 // 弃用一条边（确定不选它）。已经定了的边再否一次是幂等的，不算错：
131 // 传播里可能有多条路径否到同一条边，重复处理不该被当成冲突。
132 bool Solver::banEdge(int e) {
133     if (st_[e] != 0) return true;
134     trail_.push_back({K_STATE, e, st_[e]});
135     st_[e] = 2;
136     // 弃用的边等于从两个端点的"可选边"里拿掉，两端的自由度都要减。
137     subFree(edges_[e].u);
138     subFree(edges_[e].v);
139     return true;
140 }
141
142 // 选中一条边，并顺手处理它带来的两条后果：
143 //   (a) 两端各消耗一个度；
144 //   (b) 它横跨的空格里，所有对向（竖直）的候选边都不能再选了，因为栅栏不能交叉。
145 // 返回 false 表示这个选择立刻导致矛盾，调用方要回滚。
146 //
147 // 第 (b) 条是"双向"的：如果某个空格里已经躺着一条竖边，这里返回 false；
148 // 否则把那条竖边弃掉，它两侧的点自由度减一，可能又触发新的连锁。
149 // 因为每个空格最多只可能有一条横边和一条竖边（边界上连一条都没有），
150 // 所以这个循环的代价是这条边的长度，与度数无关。
151 bool Solver::selectEdge(int e) {
152     if (st_[e] == 1) return true;   // 已经选过，幂等
153     if (st_[e] == 2) return false;  // 这条边此前已经被否掉，不能再选
154
155     trail_.push_back({K_STATE, e, st_[e]});
156     st_[e] = 1;
157     const Edge& ed = edges_[e];
158     if (!addNeed(ed.u, -1) || !addNeed(ed.v, -1)) return false;
159     subFree(ed.u);
160     subFree(ed.v);
161
162     // 注意这里查的是"相交的边"而不是"相交的格子"。同一格里最多只有一条横边和
163     // 一条竖边，所以按格子能直接定位到唯一那条要否的边；如果对向边已经被选中，
164     // 说明两条线在这个空格里十字交叉了，直接冲突。
165     if (ed.r1 == ed.r2) {
166         const int lo = std::min(ed.c1, ed.c2), hi = std::max(ed.c1, ed.c2);
167         for (int c = lo + 1; c < hi; ++c) {
168             const int o = vEdge_[ed.r1 * m_ + c];
169             if (o < 0) continue;
170             if (st_[o] == 1) return false;  // 已经有竖线横在这里了
171             if (!banEdge(o)) return false;
172         }
173     } else {
174         const int lo = std::min(ed.r1, ed.r2), hi = std::max(ed.r1, ed.r2);
175         for (int r = lo + 1; r < hi; ++r) {
176             const int o = hEdge_[r * m_ + ed.c1];
177             if (o < 0) continue;
178             if (st_[o] == 1) return false;
179             if (!banEdge(o)) return false;
180         }
181     }
182     return true;
183 }
184
```

```
// 回滚到 trail_ 长度等于 mark 的那一刻。倒着还原是因为同一个量可能被改过多次，
// 只有按相反顺序覆盖回去，最终值才是第一次改动之前的值。
void Solver::undo(size_t mark) {
    while (trail_.size() > mark) {
        const Change ch = trail_.back();
        trail_.pop_back();
        if (ch.kind == K_STATE) {
            st_[ch.idx] = (char)ch.old;
        } else if (ch.kind == K_NEED) {
            need_[ch.idx] = ch.old;
        } else {
            freeCnt_[ch.idx] = ch.old;
        }
    }
    // 队列里剩下的项属于刚被撤销的那一层，状态已经回去了，不能再按它们检查。
    // 这里直接清空而不是逐项 unflag：清队列的同时把 inQueue_ 一起清掉，
    // 否则下次 pushVertex 会以为这个点还在队列里，白白漏掉一次传播。
    for (int v : queue_) inQueue_[v] = 0;
    queue_.clear();
}

// 为什么用"改动日志"而不是"进入分支时整份拷贝"：
// 状态一共 V+E 个字段（V ≤ 1200，E ≤ 5000），一份拷贝就是几万个字节，
// 每个搜索结点拷一次，几十万结点就是几十 GB 的挪动，光 memcpy 就能吃掉全部预算。
// trail 只记真正被改过的字段，一次传播通常只有几十条记录，失败时按倒序还原即可，
// 代价与"改了多少"成正比，而不是与问题规模成正比。
// 代价是 undo 必须严格倒序——同一个量可能在一层里被改多次，
// 顺序错了就会还原成中间值。所以所有状态修改都只走 selectEdge/banEdge/addNeed/subFree
// 这四个入口，别的函数不许直接写 st_/need_/freeCnt_。
//
// 约束传播：一直处理队列，直到没有新的点需要检查。
// 三条规则（都在"只做必然的推断"范围内）：
//   need < 0 或 need > free  -> 这个点无论如何凑不出 k 条边，整支失败；
//   need == 0                -> 这个点已经够了，剩下的未定边一律弃用；
//   need == free             -> 剩下的边一条都不能少，全部选中。
// 注意规则是"链式"的：选一条边会减少两端的需求，可能让邻居也满足后两条规则，
// 所以要用队列反复推，而不是扫一遍就完。
bool Solver::propagate() {
    while (!queue_.empty()) {
        const int v = queue_.back();
        queue_.pop_back();
        inQueue_[v] = 0;
        ++propagations_;

        if (need_[v] < 0 || need_[v] > freeCnt_[v]) return false;

        if (need_[v] == 0) {
            // 这个点已经够了，所有还没定的关联边一律不能选。
            for (int e : inc_[v]) {
                if (st_[e] == 0 && !banEdge(e)) return false;
            }
        } else if (need_[v] == freeCnt_[v]) {
            // 剩下的边一条都不能少，全选上。
            for (int e : inc_[v]) {
                if (st_[e] == 0 && !selectEdge(e)) return false;
            }
```

```
241         }
242     }
243     return true;
244 }
245
246 // 挑分支点。四种策略只影响搜索快慢，不影响答案对不对（第 4 章有实测对比）：
247 //   0 自由度减需求最小：最接近被逼死的点，分支最少（默认，也是实测最好的）
248 //   1 行优先第一个：最朴素，写起来只要一行
249 //   2 需求最大：先安排度数高的点
250 //   3 未定边最多：先安排选择最多的点
251 // 0 和 2/3 的优化方向相反，所以两类分数分开记：bestSlack/bestFree 越小越好，
252 // bestScore 越大越好。返回 -1 表示所有点的需求都已归零，可以收工。
253 int Solver::chooseVertex() const {
254     int best = -1;
255     int bestSlack = 1 << 30;
256     int bestFree = 1 << 30;
257     int bestScore = -1;
258     for (int v = 0; v < (int)need_.size(); ++v) {
259         if (need_[v] == 0) continue;
260         if (heur_ == 1) return v;
261         if (heur_ == 2) {
262             if (need_[v] > bestScore) { best = v; bestScore = need_[v]; }
263             continue;
264         }
265         if (heur_ == 3) {
266             if (freeCnt_[v] > bestScore) { best = v; bestScore = freeCnt_[v]; }
267             continue;
268         }
269         const int slack = freeCnt_[v] - need_[v];
270         if (slack < bestSlack || (slack == bestSlack && freeCnt_[v] < bestFree)) {
271             best = v;
272             bestSlack = slack;
273             bestFree = freeCnt_[v];
274         }
275     }
276     return best;
277 }
278
279 // 回溯主体：先传播，再挑一个点分支，枚举"选其中 need 条"的所有组合。
280 // 完备性：任何合法解在这个点上选的就是某个 size 恰为 need 的子集，所以枚举不会漏解。
281 // 剪枝：每个组合先 select/ban 再递归，递归里第一件事又是传播，冲突会尽早暴露。
282 //
283 // 代价：每个结点的开销是一次 O(V) 的 chooseVertex 加若干次传播，
284 // 分支宽度最多 C(4,2)=6，所以结点数就是主要成本。实测最坏的一类输入是
285 // 50×50、密度 0.6 的坑位（大量 k=3/k=4 的点互相牵连，传播推不动），
286 // 单个结点约 0.5 μs，跑到 40 万结点时接近 0.2 s；再密就会指数爆炸，
287 // 这也正是第 4 章要讨论的地方。
288 bool Solver::dfs() {
289     ++nodes_;                          // 结点数只用来做实验统计，不参与逻辑
290     if (useProp_ && !propagate()) return false;
291
292     const int v = chooseVertex();
293     if (v < 0) return true;   // 所有连接器的 need 都归零，方案成立
294
295     // 这个点最多四度，把还没定的关联边摊开，枚举"选其中 need 条"的所有组合。
296     int cand[4];
```

```
    int cnt = 0;
    for (int e : inc_[v]) {
        if (st_[e] == 0) cand[cnt++] = e;
    }
    const int needHere = need_[v];
    const size_t mark = trail_.size();

    for (int mask = 0; mask < (1 << cnt); ++mask) {
        if (__builtin_popcount((unsigned)mask) != needHere) continue;

        bool ok = true;
        for (int i = 0; i < cnt && ok; ++i) {
            ok = (mask >> i & 1) ? selectEdge(cand[i]) : banEdge(cand[i]);
        }
        if (ok && dfs()) return true;
        undo(mark);   // 这一组组合失败，退回到分支前的状态试下一组
    }
    return false;
}

// 把选中的边整理成题目要的方向表：第 i 项是第 i 个连接器的 {上,下,左,右}。
// 连接器本身就按行优先编号，所以这个数组的顺序就是输出的顺序。
std::vector<std::array<int, 4>> Solver::directions() const {
    std::vector<std::array<int, 4>> dirs(cellOf_.size(), {0, 0, 0, 0});
    for (int v = 0; v < (int)cellOf_.size(); ++v) {
        const int r = cellOf_[v] / m_;
        const int c = cellOf_[v] % m_;
        for (int e : inc_[v]) {
            if (st_[e] != 1) continue;
            // 边的另一端在哪一侧，就说明这个连接器往哪个方向有一段栅栏。
            // 注意两端之间可能隔着好几个空格，所以不能拿"差一格"来判断方向，
            // 只能先看它是横边还是竖边，再比较大小。（这里踩过坑：曾经用行差 ±1
            // 判上下，结果跨多格的竖边被误判成"右"，样例 1 的第 6 列直接少一位。）
            const Edge& ed = edges_[e];
            const int orow = (ed.u == v) ? ed.r2 : ed.r1;
            const int ocol = (ed.u == v) ? ed.c2 : ed.c1;
            if (orow == r) {
                if (ocol < c) dirs[v][2] = 1;
                else dirs[v][3] = 1;
            } else {
                if (orow < r) dirs[v][0] = 1;
                else dirs[v][1] = 1;
            }
        }
    }
    return dirs;
}

// 入口：清状态、做两条一眼能看出的无解判定、然后把所有点排进队列开始搜索。
// 两条判定都很便宜，命中一次就省掉整棵搜索树：
//   * 某个点的度数比它连边数还大（周围根本没有那么多邻居）；
//   * 总度数是奇数（每条边贡献 2 度，奇数永远凑不出来）。
//
// 规模：n,m ≤ 50 时 V=点数 ≤ 1200（同行同列相邻对的上限），E ≤ 2nm = 5000，
// 所以下面所有"逐点扫一遍"的操作在常数上都只相当于几万次访存，
// 缓存全放得下；真正决定运行时间的是搜索树结点数，不是单个结点的开销。
```

```
bool Solver::solve() {
    const int V = (int)cellOf_.size();

    st_.assign(edges_.size(), 0);
    need_.assign(V, 0);
    freeCnt_.assign(V, 0);
    queue_.clear();
    inQueue_.assign(V, 0);
    trail_.clear();
    nodes_ = 0;
    propagations_ = 0;

    long long totalNeed = 0;
    for (int v = 0; v < V; ++v) {
        need_[v] = deg_[cellOf_[v]];
        freeCnt_[v] = (int)inc_[v].size();
        if (need_[v] > freeCnt_[v]) return false;   // 这个点连出边都不够
        totalNeed += need_[v];
    }
    // 每条选中的边给总度数贡献 2，所以总度数是奇数时一定无解。
    // 这一条能在建完图后立刻否掉一批数据，省掉整棵搜索树。
    if (totalNeed % 2 != 0) return false;

    for (int v = 0; v < V; ++v) pushVertex(v);
    return dfs();
}

}  // namespace p3
```

## A.3 main.cpp

```
// main.cpp —— 读入、调用求解器、按题目要求的格式输出
//
// 输入：第一行 n m；接下来 n 行、每行 m 个整数，0 表示空格，1..4 表示该格的连接器
//       还要接几条栅栏。
// 输出：无解打一行 "No Solution"；有解则每个连接器一行：
//       row col up down left right
//       row/col 从 1 开始，四个方向位是 0/1，按行优先顺序输出。
//
// 拿题面样例 3 举例（3 行 5 列），输入是
//       3 5
//       1 1 2 0 1
//       1 0 3 0 2
//       1 2 1 0 1
// 输出是 11 行 6 列的整数，每行对应一个连接器，例如 "1 1 0 0 0 1" 表示
// 第 1 行第 1 列那个连接器往右有一段栅栏（上下左都没有）。多解时给哪个都算对。
//
// 命令行开关（只影响实验，不影响判题输出）：
//   --stats        往 stderr 打一行 nodes= propagate= time_ms= answer=，
//                  stdout 的答案保持原样，所以带 --stats 也能直接拿去对拍。
//   --heur N       换分支点策略（0..3，见 solver.h）；默认 0 是实测最稳的那种。
//   --noprop       关掉传播，只留逐边检查；用来量传播值多少，判题时不会用到。
#include <chrono>
#include <cstdio>
#include <cstdlib>
```

```cpp
#include <cstring>
#include <vector>

#include "solver.h"

namespace {

// 题目的输入小得很（最多 50×50=2500 个数），用 scanf 读完全够。
// 这里不做任何合法性检查之外的补救：读失败就报错退出，免得把空输入当成"无解"。
bool readInput(int& n, int& m, std::vector<int>& grid) {
    if (std::scanf("%d %d", &n, &m) != 2) return false;
    if (n <= 0 || m <= 0) return false;
    grid.assign((size_t)n * m, 0);
    for (int i = 0; i < n * m; ++i) {
        if (std::scanf("%d", &grid[i]) != 1) return false;
    }
    return true;
}

}  // namespace

int main(int argc, char** argv) {
    bool stats = false;
    bool prop = true;
    int heur = 0;
    // 只认这几个开关，其余参数一律忽略：判题机上只会重定向 stdin，不会有参数。
    for (int i = 1; i < argc; ++i) {
        if (std::strcmp(argv[i], "--stats") == 0) {
            stats = true;
        } else if (std::strcmp(argv[i], "--heur") == 0 && i + 1 < argc) {
            heur = std::atoi(argv[++i]);
        } else if (std::strcmp(argv[i], "--noprop") == 0) {
            prop = false;
        }
    }

    int n = 0, m = 0;
    std::vector<int> grid;
    if (!readInput(n, m, grid)) {
        std::fprintf(stderr, "usage: recover [--stats] [--heur N] [--noprop] < input\n");
        return 1;
    }

    p3::Solver solver;
    solver.init(n, m, grid);
    solver.setHeuristic(heur);
    solver.setPropagation(prop);

    // 计时只包住 solve()：读入和输出的时间不算进去，
    // 否则报告里的表会被 printf 的 2 万多次调用污染。
    const auto t0 = std::chrono::steady_clock::now();
    const bool ok = solver.solve();
    const auto t1 = std::chrono::steady_clock::now();

    if (!ok) {
        std::printf("No Solution\n");
```

```
    } else {
        // 连接器编号本来就是行优先的，直接按编号顺序输出就是题目要的顺序；
        // 坐标加 1 是因为题面用 1 基，内部全用 0 基。
        const auto dirs = solver.directions();
        const auto& cells = solver.connectorCell();
        for (size_t i = 0; i < cells.size(); ++i) {
            const int r = cells[i] / m + 1;
            const int c = cells[i] % m + 1;
            std::printf("%d %d %d %d %d %d\n", r, c, dirs[i][0], dirs[i][1], dirs[i][2], dirs[i
    ][3]);
        }
    }

    if (stats) {
        // 三个数都是给实验用的：nodes 是 dfs 调用次数，propagate 是传播里检查过的点数，
        // time_ms 用 steady_clock 算墙钟（不受系统时间调整影响）。
        // 这些都走 stderr，所以带 --stats 跑出来的 stdout 仍然是干净答案，能直接喂给校验脚本。
        const double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
        std::fprintf(stderr, "nodes=%lld propagate=%lld time_ms=%.3f answer=%s\n",
                     solver.nodes(), solver.propagations(), ms, ok ? "yes" : "no");
    }
    return 0;
}
```